

SELECT 문의 기본 문법

ch03.sql 코드를 실행 순서대로 분석한 학습자료입니다. 각 예제는 EMPLOYEES 테이블 (HR 스키마) 기준이며, 개념 → 문법 → 코드 → 주의사항 순으로 정리했습니다.

00 도입 예제 — JOIN 미리보기

파일 맨 앞에 등장하는 코드는 아직 배우지 않은 **JOIN + 서브쿼리** 예제입니다. "SELECT의 결과가 다음 단계에서 FROM절의 테이블처럼 쓰일 수 있다"는 것을 미리 보여주는 티저 역할로, 3장 본문 순서상 실제 학습은 아래 1번부터 시작됩니다.

```
SELECT *
FROM employees A
JOIN (
    SELECT * FROM departments
    WHERE department_id = 20
) B
ON A.department_id = B.department_id;
```

강의 포인트 — 이 시점에서는 "괄호 안 SELECT가 또 하나의 테이블처럼 동작한다"는 정도만 언급하고, 서브쿼리(7장)와 JOIN(6장)에서 자세히 다룰 것을 예고하는 용도로 사용하면 좋습니다.

01 SELECT 문의 기본 문법

기본 전체 컬럼 / 특정 컬럼 조회

* 은 모든 컬럼을, 컬럼명을 콤마로 나열하면 필요한 컬럼만 선택 조회합니다.

```
SELECT 컬럼1, 컬럼2, ... | *  
FROM 테이블명;
```

```
select *  
from employees;  
  
select employee_id, first_name, last_name  
from employees;
```

실무 팁 — 운영 쿼리에서는 SELECT * 대신 필요한 컬럼만 명시하는 것이 네트워크 비용·가독성 측면에서 유리합니다. 학습 단계에서는 구조 파악용으로 *를 먼저 씁니다.

02 ORDER BY — 정렬

조회 결과의 정렬 순서를 지정합니다. 기본값은 오름차순(ASC)이라 생략 가능하며, 내림차순은 DESC를 명시해야 합니다.

```
SELECT 컬럼 FROM 테이블  
ORDER BY 컬럼 [ASC|DESC];
```

```
select employee_id, first_name, last_name  
from employees  
order by employee_id desc; -- asc는 생략 가능(기본 오름차순) / desc는 명시 필수(내림차순)
```

키워드	의미	생략 가능 여부
ASC	오름차순 (작은 값 → 큰 값)	✅ 생략 시 기본값
DESC	내림차순 (큰 값 → 작은 값)	❌ 반드시 명시

지정한 컬럼(들) 기준으로 중복된 행을 제거하고 유일한 값만 반환합니다.

```
SELECT DISTINCT 컬럼 FROM 테이블 ORDER BY ...;
```

```
select distinct job_id
from employees
order by 1 desc;
```

ORDER BY 1의 의미 — 컬럼명 대신 SELECT절에 나열된 순번을 쓸 수 있습니다. 1은 첫 번째 선택 컬럼(job_id)을 뜻합니다. 컬럼이 여러 개일 때 짧게 쓸 수 있지만, 가독성을 위해 실무에서는 컬럼명 사용을 권장합니다.

주의 — DISTINCT는 SELECT절 전체 조합 기준으로 중복을 제거합니다. 컬럼이 2개 이상이면 "그 조합"이 유일한 행만 남습니다.

조회 결과의 컬럼 헤더를 원하는 이름으로 바꿔 표시합니다. 원본 테이블 구조는 변경되지 않는 일시적 표시명입니다.

```
SELECT 컬럼 AS 별칭 (AS 생략 가능) FROM 테이블;
```

```
select employee_id as 사원번호, first_name as 이름, last_name as 성
from employees;
```

팁 — AS는 생략 가능(employee_id 사원번호)하지만, 가독성을 위해 명시하는 스타일을 권장합니다. 별칭에 공백/한글을 포함하려면 큰따옴표로 감쌉니다: "사원 번호".

05 문자열 연결 연산자 (||)

파이프 두 개(||)로 여러 컬럼·리터럴 문자열을 하나로 이어붙입니다.

컬럼1 || 컬럼2 || '리터럴' AS 별칭

```
select employee_id, first_name||last_name as 성명
from employees;
```

```
select employee_id,
       first_name||last_name as 성명,
       email||'@'||'walmart.com' as 이메일
from employees;
```

구조 분석 — email||'@'||'walmart.com'은 컬럼 값과 문자열 리터럴 2개를 순서대로 이어붙여 steven.king@walmart.com 형태의 가짜 이메일을 생성하는 예제입니다.

06 산술 연산 처리

SELECT절에서 숫자 컬럼에 대해 사칙연산을 바로 적용할 수 있습니다. 연산자 우선순위는 일반 수학과 동일(*, / 먼저, 괄호 최우선)합니다.

```
select employee_id as 사원번호,
       salary as 급여,
       salary+500 as 추가급여,
       salary-100 as 인하급여,
       (salary*1.1)/2 as 조정급여
from employees;
```

연산자	의미
+ -	덧셈 / 뺄셈
* /	곱셈 / 나눗셈 (우선순위 높음)
()	괄호 — 연산 순서 강제

조건을 만족하는 행만 필터링합니다.

```
select * from employees where employee_id = 100;
select * from employees where first_name = 'David';
select * from employees where employee_id >= 105;
```

연산자	의미
=	같음
> / >=	초과 / 이상
< / <=	미만 / 이하

주의 — 문자열 비교(first_name = 'David')는 **대소문자를 구분**합니다. 'david'로 쓰면 결과가 없습니다.

두 값 사이(양 끝값 포함)를 조건으로 지정합니다.

컬럼 BETWEEN 최소값 AND 최대값

```
select *
from employees
where salary BETWEEN 10000 AND 20000
order by salary desc;
```

동치 표현 — salary BETWEEN 10000 AND 20000은 salary >= 10000 AND salary <= 20000과 완전히 같습니다. 양 끝값이 포함되는 것이 핵심입니다.

여러 후보값 중 하나라도 일치하면 조건을 만족합니다. OR 여러 개를 대체합니다.

컬럼 IN (값1, 값2, 값3, ...)

```
select *
from employees
where salary in (10000, 17000, 24000);
```

동치 표현 — 위 조건은 salary=10000 OR salary=17000 OR salary=24000과 동일하지만 훨씬 간결합니다. 서브쿼리 결과를 그대로 IN의 값 목록으로 넣는 형태(7장)로 확장됩니다.

와일드카드를 이용해 일부만 일치하는 문자열을 검색합니다.

와일드카드

의미

%

0개 이상의 임의 문자

_

정확히 1개의 임의 문자

```
-- 직원의 직책이 'AD'로 시작하는 모든 행
select * from employees
where job_id like 'AD%';

-- job_id가 'AD' + 정확히 3글자 (총 5자리)
select * from employees
where job_id like 'AD___'; -- 언더바 3개 = 3글자 매칭
```

흔한 실수 — 'AD%'와 'AD___'는 결과가 다릅니다. %는 길이 제한이 없고, _는 글자 수를 정확히 고정합니다.

11 IS NULL / IS NOT NULL

NULL은 "값이 없음"을 뜻하는 특수 상태로, `= NULL` 로는 비교할 수 없고 반드시 `IS NULL` 전용 연산자를 사용해야 합니다.

```
select * from employees
where manager_id is null; -- ❌ manager_id = null (항상 결과 없음, 오답 패턴)
```

```
select * from employees
where manager_id is not null;
```

핵심 개념 — NULL은 "알 수 없음"의 의미라 `NULL = NULL` 조차 결과가 UNKNOWN(참이 아님)입니다. 그래서 `=`이 아닌 `IS` 키워드로만 판별합니다. 최상위 관리자(사장)의 `manager_id`가 NULL인 대표 사례입니다.

12 논리 연산자 — AND / OR / NOT

여러 조건을 결합합니다. 우선순위는 `NOT > AND > OR` 순이며, 의도한 순서와 다를 경우 괄호로 명확히 묶어야 합니다.

```
select * from employees
where salary > 4000 and job_id = 'IT_PROG';
```

```
-- 괄호로 OR 조건을 명확히 묶은 케이스
select * from employees
where salary > 4000
and (job_id = 'IT_PROG' or job_id = 'FI_ACCOUNT');
```

괄호가 없다면? — `salary>4000 and job_id='IT_PROG' or job_id='FI_ACCOUNT'`로 쓰면 AND가 OR보다 먼저 묶여 "salary>4000이면서 IT_PROG" **이거나** "무조건 FI_ACCOUNT"로 의미가 바뀝니다. 급여 조건 없이 FI_ACCOUNT 전원이 포함되는 의도치 않은 결과가 나올 수 있습니다.

13 부정 비교 연산자 (<>, !=)

"같지 않음"을 표현하는 두 가지 표기법으로, Oracle에서는 완전히 동일하게 동작합니다.

```
select * from employees where employee_id <> 105;  
select * from employees where employee_id != 105;
```

표준 표기 — ANSI SQL 표준은 <>이며, !=는 여러 DBMS에서 지원하는 비표준 관용 표기입니다.
Oracle은 둘 다 지원하므로 팀 컨벤션에 맞춰 통일해서 사용하면 됩니다.

✓ 확인 문제 (셀프 체크)

① 급여(salary)가 8000 이상 12000 이하인 직원을 **급여 내림차순**으로 조회하시오.

② job_id가 **'SA_'**로 시작하는 직원의 사원번호, 이름, job_id를 조회하시오.

③ manager_id가 없는(NULL) 직원의 이름과 부서번호를 조회하시오.

④ job_id가 **'IT_PROG'** 또는 **'SA_REP'** 이면서 급여가 5000을 초과하는 직원을 조회하시오. (괄호 사용 필수)

정답 힌트 — 1번은 BETWEEN+ORDER BY DESC, 2번은 LIKE 'SA_%' (언더바는 리터럴이 아닌 와일드카드이므로 이스케이프 필요할 수 있음, 또는 LIKE 'SA_%' ESCAPE '\'), 3번은 IS NULL, 4번은 AND (...OR...) 조합으로 풀이합니다.